

06 — Memória como Modelo de Custo

Pré-requisito: [05-latency-vs-bandwidth.md](#)

1. Intuição: você provavelmente aprendeu a contar o "custo" errado

Quando alguém aprende algoritmos pela primeira vez, a métrica de custo padrão é **número de operações** — quantas somas, multiplicações, comparações um algoritmo faz (isso é o que notação Big-O geralmente conta). Essa métrica fez sentido historicamente, quando CPUs eram lentas pra calcular e memória era relativamente rápida em comparação.

Esse mundo não existe mais. Depois do que você viu no arquivo 04 e 05, já dá pra sentir o problema: se um acesso à RAM custa ~100 ciclos e uma soma de ponto flutuante custa ~1 ciclo, **contar operações aritméticas como se fossem o custo dominante está errado na maioria dos casos reais**. O custo real, na maior parte do código moderno, é **mover dado**, não calcular com ele.

2. Um exemplo concreto pra sentir isso

```
void escala(float* data, int n, float fator) {  
    for (int i = 0; i < n; i++) {  
        data[i] = data[i] * fator;  
    }  
}
```

Por elemento, essa função faz:

- **1 leitura** de `data[i]` da memória (4 bytes)
- **1 multiplicação** (a operação aritmética em si)
- **1 escrita** de volta pra `data[i]` (4 bytes)

Custo "clássico" (contando operações): 1 multiplicação por elemento — parece muito barato.

Custo real de memória: 8 bytes movidos por elemento (4 lidos + 4 escritos), pra **apenas uma** operação aritmética feita com esse dado. Isso é uma proporção péssima de cálculo por byte movido — e é exatamente o número que o próximo arquivo (arithmetic intensity) vai formalizar.

3. Reformulando a pergunta de "quanto trabalho" pra "quanto dado se move"

O modelo de custo deste módulo (e de praticamente todo trabalho sério de performance numérica) não é "quantas operações". É:

Quantos bytes precisam entrar na unidade de cálculo,
e quantos bytes precisam sair dela,
para realizar esse trabalho?

Essa reformulação não é estética — ela muda completamente que otimizações fazem sentido. Se o gargalo é mover dado, **fazer mais cálculo com o mesmo dado já movido** pode ser de graça (a ULA está ociosa enquanto espera memória de qualquer jeito — lembra do latency hiding do arquivo 05). Mas **mover menos dado** — mesmo que isso signifique recalcular algo em vez de guardar — pode ser a otimização real.

Isso não é abstrato pra você: é literalmente o trade-off de **save vs recompute** que aparece explicitamente no [research/north-star.md](#) do seu próprio lab, sobre kernels de normalização — a pergunta "vale mais guardar um valor intermediário (custa memória) ou recalcular ele (custa cálculo)?" só faz sentido quando você já parou de contar operações e começou a contar bytes.

PERGUNTA RÁPIDA 1: entre duas versões de uma função que calculam exatamente o mesmo resultado — uma que faz 10 operações aritméticas e move 100 bytes, outra que faz 50 operações aritméticas e move 20 bytes — qual delas você esperaria que rodasse mais rápido num hardware moderno, tipicamente?

► Resposta

4. Por que isso vira uma fórmula no próximo arquivo

Contar "bytes movidos" e "operações feitas" separadamente já é útil, mas a pergunta que realmente importa é a **proporção** entre os dois — quanto cálculo você consegue "espremer" de cada byte que precisou ser movido. Essa proporção tem nome e fórmula: **arithmetic intensity**, o assunto do arquivo 07. Tudo que você viu aqui é a preparação conceitual pra essa fórmula fazer sentido de imediato, em vez de parecer arbitrária.

5. Por que isso importa para CUDA

GPUs têm capacidade de cálculo bruta **enorme** comparada à sua bandwidth de memória disponível (essa proporção específica é o que o modelo Roofline, arquivo 09, formaliza). Isso significa que, em GPU, "mover menos dado" quase sempre importa mais que "fiz menos contas" — o oposto da intuição que vem de contar operações do jeito clássico. Times de performance de kernel gastam a maior parte do esforço reduzindo tráfego de memória, não reduzindo FLOPs — é literalmente o objeto de estudo do North Star do seu lab (kernels de normalização são memory-bound; a pergunta inteira é sobre reduzir tráfego, não reduzir cálculo).

6. Erros comuns

- Continuar otimizando "número de operações" quando o gargalo real é memória — esforço gasto no lugar errado.
- Achar que "fazer mais cálculo" é sempre ruim. Se a ULA está ociosa esperando dado de qualquer forma (memory-bound), cálculo extra pode ser essencialmente grátis.
- Ignorar que essa análise depende do **hardware específico** — a proporção entre velocidade de cálculo e velocidade de memória muda entre CPU e GPU, e entre gerações de GPU. Não existe resposta universal, só a proporção medida no seu dispositivo (por isso o deliverable deste módulo exige medição real, não número de datasheet).

PREDICT BEFORE RUNNING (conceitual)

Considere três operações sobre um array de n floats:

A) $out[i] = a[i] + b[i]$	(1 soma, lê 2 floats, escreve 1)
B) $out[i] = a[i] * a[i] + b[i] * b[i] + c[i]$	(4 operações, lê 3 floats, escreve

1)

C) `out[i] = sin(a[i]) * cos(a[i])`

(2 operações caras + lê 1, escreve

1)

1. Ordene as três por "bytes movidos por elemento" (do menor pro maior).
2. Ordene as três por "operações aritméticas por elemento" (do menor pro maior).
3. As duas ordens são iguais? O que isso já sugere sobre qual operação é mais provável de ser memory-bound vs compute-bound?

► Explicação